

INSTALACIÓN DE REACT NAVIGATION

Proyecto Login / SignUp | Desarrollo Móvil | ISDEM N° 8207

Antes de empezar: asegúrense de tener el proyecto ya creado y funcionando siguiendo el instructivo "Instalación del entorno de desarrollo" (proyecto Expo con plantilla blank en JavaScript). Todos los comandos siguientes se ejecutan desde la terminal, parados dentro de la carpeta del proyecto (por ejemplo, mobileStart).

Instalación de la librería principal

Para comenzar, es necesario instalar la librería principal de React Navigation, la cual constituye la base del sistema de navegación en aplicaciones con React Native. Para implementarla en su proyecto ejecuten:

```
npx expo install @react-navigation/native
```

Tipos de navegación

Stack Navigation

Permite a los usuarios navegar entre pantallas de forma secuencial, como si se apilaran una sobre otra. Cada nueva pantalla se coloca encima de la anterior, y el usuario puede volver utilizando un botón de retroceso. El comando para su uso es:

```
npm install @react-navigation/stack
```

Bottom Tabs Navigation

Permite a los usuarios cambiar entre distintas pantallas mediante pestañas ubicadas generalmente en la parte inferior de la aplicación. Es útil para apps que requieren acceso rápido a varias secciones principales. Se aplica mediante la siguiente instrucción:

```
npm install @react-navigation/bottom-tabs
```

Modal Navigation

Se utiliza para mostrar contenido adicional sin interrumpir el flujo de navegación principal. Generalmente aparece como una ventana emergente sobre la pantalla actual. Es ideal para formularios, alertas o información adicional. Este tipo de navegación se incluye dentro del comando de Stack Navigation, no requiere instalación aparte.

Drawer Navigation

Permite acceder a diferentes secciones de la aplicación mediante un menú lateral que se desliza desde uno de los lados de la pantalla. Es útil para aplicaciones con múltiples secciones o configuraciones. La implementación se efectúa con la siguiente orden:

```
npm install @react-navigation/drawer
```

Deep linking

Permite abrir una aplicación en una pantalla específica a través de una URL. Es útil para enviar a los usuarios a contenido específico dentro de la aplicación desde otros lugares, como enlaces web, correos electrónicos o mensajes.

No requiere un comando de instalación específico, pero sí configuración. Para implementar este tipo de navegación en nuestro proyecto deben consultar su correspondiente documentación en: <https://reactnavigation.org/docs/deep-linking/>.

Paquetes necesarios para el funcionamiento de la navegación

`react-native-screens` optimiza el uso de memoria al manejar las pantallas. Las pantallas inactivas se liberan de la memoria nativa, reduciendo el consumo de recursos y mejorando el rendimiento de la aplicación.

`react-native-safe-area-context` permite que el contenido de la aplicación se adapte a las "zonas seguras" de la pantalla, evitando superposiciones con elementos del sistema operativo como el notch, la barra de estado o la barra de navegación.

Para instalarlos, ejecuten:

```
npx expo install react-native-screens react-native-safe-area-context
```

Nota: usen siempre `npx expo install` (en vez de `npm install`) para las librerías nativas como `react-navigation/native`, `screens`, `safe-area-context` o `vector-icons`. Ese comando elige automáticamente la versión de cada paquete compatible con el SDK de Expo que están usando, evitando problemas de compatibilidad entre versiones.

Instalación de paquetes adicionales

Instalen el SDK de Firebase para agregar funcionalidades como autenticación, bases de datos en tiempo real y almacenamiento en la nube:

```
npm install firebase
```

Para utilizar íconos en su aplicación (por ejemplo, en pestañas o botones) desde Vector Icons, instalen el siguiente paquete compatible con Expo:

```
npx expo install @expo/vector-icons
```

Estructura de carpetas y archivos

Creen la carpeta **navigation** y dentro el archivo **Navigation.js**. Esta carpeta y archivo centralizarán la configuración del sistema de navegación de la aplicación, promoviendo una mejor modularidad, organización y mantenibilidad del código.

Luego, creen la carpeta **screens** con los archivos **Login.js**, **SignUp.js** y **Home.js**. Esta carpeta contendrá los componentes principales que representan cada pantalla de la aplicación. Separar cada pantalla en su propio archivo nos ayudará a mantener una arquitectura limpia, modular y escalable.

Por último, creen la carpeta **src** y dentro de la misma otra denominada **config** con el archivo **firebaseConfig.js**. Esta estructura nos permite organizar configuraciones externas del proyecto (como la configuración de Firebase) de forma clara y mantenible.

El archivo `firebaseConfig.js` centraliza las credenciales y parámetros de Firebase, facilitando la gestión de servicios en la nube y manteniendo la lógica de configuración separada del resto de la aplicación.

```
mobileStart
├── .expo
├── .vscode
└── assets
```

```
|   ├── adaptive-icon.png
|   ├── favicon.png
|   ├── icon.png
|   ├── logo.png
|   └── splash-icon.png
├── navigation
|   └── Navigation.js
├── node_modules
├── screens
|   ├── Home.js
|   ├── Login.js
|   └── SignUp.js
├── src
|   ├── config
|   └── firebaseConfig.js
├── .gitignore
├── App.js
├── app.json
├── index.js
├── package-lock.json
└── package.json
```